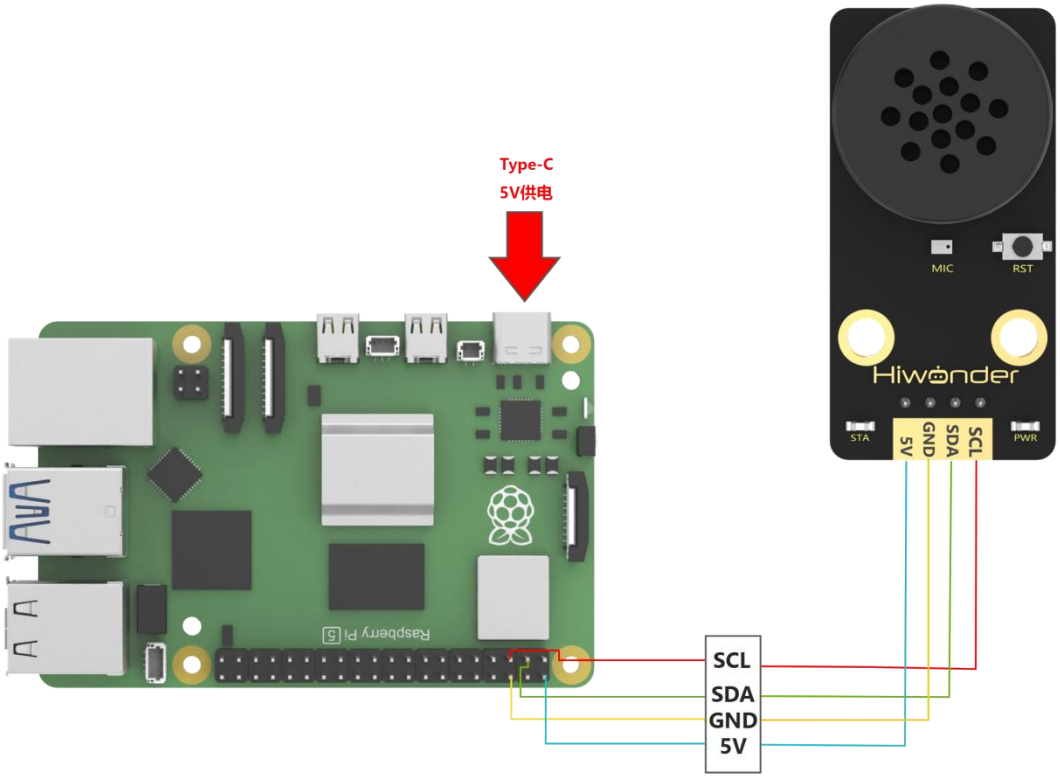
01 Raspberry Pi Module Address Modification Example

Note: This function is only supported by the new version of the voice interaction module. For detailed information on the I2C address modification feature and the differences between the old and new module versions, please refer to section 5. I2C Device Address Modification and Address Register in the document "4.1 Master-Slave Device Communication Principles."

1. Getting Started

1.1 Wiring Instruction

When wiring, connect the 5V, GND, SDA, and SCL pins of the WonderEcho voice recognition module to the corresponding pins on the Raspberry Pi controller. The wiring method is shown in the diagram below:



Note:

1. Before powering on, ensure that no metal objects are touching the controller. Otherwise, the exposed pins at the bottom of the board may cause a short circuit and damage the controller.

2. The IIC pin layout of Raspberry Pi 5 is identical to that of Raspberry Pi 4, so the wiring method described here also applies to Raspberry Pi 4.

1.2 Importing and Running the Program

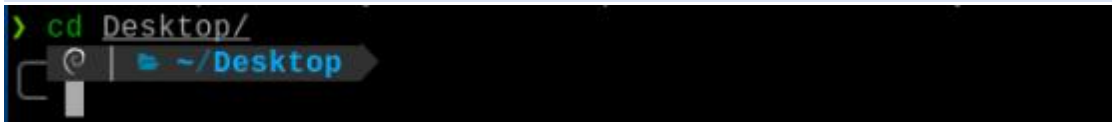
Note: For detailed instructions on installing and using MobaXterm and VNC Viewer, refer to "4.6.1 Remote Connectivity and File Transfer\01 Remote Connectivity and File Transfer."

1) You can transfer the **asr_IICaddr_change.py** script located in the "**02 Program**" folder under the same directory as this document to the Raspberry Pi using **MobaXterm**. For example, drag and drop the file onto the Raspberry Pi desktop. Then, connect to the Raspberry Pi via **VNC Viewer**.

2) In the Raspberry Pi system, press **Ctrl+Alt+T** to open a terminal window.

3) Enter the following command to navigate to the directory where the program is located:

```
cd Desktop/
```



4) Then, run the example program with the following command:

```
python3 asr_IICaddr_change.py
```

2. Test Case

This example demonstrates how to use Raspberry Pi controller to repeatedly switch the IIC slave address of the voice interaction module and control the module to perform voice broadcasting via IIC communication using the new address.

2.1 Project Outcome

The Raspberry Pi controller controls the voice interaction module to switch its I2C address to 0x34, and then uses this new address to sequentially perform the voice broadcasting "**Going straight**" and "**Recyclable waste**".
Next, the Arduino switches the voice interaction module's slave address to 0x33, then uses this IIC address to sequentially perform the voice broadcasting "**Going straight**" and "**Recyclable waste**". This process repeats in a loop.

2.2 Program Brief Analysis

1) The program imports the `smbus` library for IIC communication and the `time` library for time-related functions, enabling communication with the voice interaction module.

```
#!/usr/bin/python3
# coding=utf8
import smbus
import time
```

2) An object named `asr_module` is instantiated from the ASRModule class. During initialization, the default IIC address of the integrated voice interaction module is passed in to establish communication between the Raspberry Pi's IIC bus and the module.

```
if __name__ == "__main__":
    asr_module = ASRModule(I2C_ADDR)
```

3) Within the `ASRModule` class, during initialization (i.e., when the class is instantiated), an internal member `bus` is created by instantiating the `SMBus` class from the smbus library. This object connects to IIC Bus 1 of the Raspberry Pi and specifies the IIC slave address passed in during initialization.

```
class ASRModule:
    def __init__(self,address, bus=1):
        # Initialize I2C bus and device address (初始化 I2C 总线和设备地址)
        self.bus = smbus.SMBus(bus)  # Use I2C bus 1 (使用 I2C 总线 1)
        self.address = address  # Device I2C address (设备的 I2C 地址)
        self.send = [0, 0]  # Initialize the array for sending data (初始化发送数据的数组)
```

4) In this section, you need to write a new IIC slave address into the IIC address register of the module so that it can operate under the new address on the IIC bus. To achieve this, you will first define a function named `wire_write_data_array()` within the `ASRModule` class, which handles writing data to the slave device over IIC. This function requires three parameters: the target register address `reg`, a list of values to write `val`, and the number of bytes to write `length`.

Inside this function, the `bus` object, an instance of the `SMBus` class, is used to call the built-in `write_i2c_block_data()` method to write the data to the target register address on the slave device.

```
def wire_write_data_array(self, reg, val, length):
    """
    Write a byte array to a specified register (向指定寄存器写入字节数组)
    :param reg: Register address (寄存器地址)
    :param val: The byte array to write (要写入的字节数组)
    :param length: The number of bytes to write (要写入的字节数)
    :return: Return True if successful, False if failed (如果成功写入返回 True，失败返回 False)
    """
    try:
        self.bus.write_i2c_block_data(self.address, reg, val[:length])
        # Send byte array to specified register (发送字节数组到设备的指定寄存器)
        return True # Write successful (写入成功)
    except IOError:
        return False # Write failed, return False (写入失败，返回 False)
```

5) Furthermore, use the `ChangeAddr()` method provided by the `ASRModule` class to perform the actual address update operation. The function first checks whether the `new_addr` argument (the new address to be assigned) is valid—that is, whether it falls within the allowed range or types. After confirming the validity of the new address, the program places `new_addr` into a list and then calls the `wire_write_data_array` function from the same class.

The list is passed as the data buffer, `ASR_IIC_ADDR_CHANGE_ADDR (0x03)` as the target register address, and `1` as the number of bytes to write, thereby completing the data write operation.

Since the address member of the `ASRModule` class still retains the module's original I2C address at this stage, the Raspberry Pi establishes communication using the old address to transmit the new one.

Once the data is successfully written, the IIC slave address on the module is updated accordingly. Subsequently, the `address` member of the `ASRModule` class is updated to the new I2C address, completing the synchronization of the I2C address within the controller program.

```
def ChangeAddr(self, new_addr):
    """
    Send IIC address change command to the device (向设备发送IIC地址修改命令)
    :new_addr: New IIC address, optional values: 0x33, 0x34 (new_addr: 新IIC地址，可选值0x33、0x34)
    """
    if new_addr == 0x33 or new_addr == 0x34:
        self.wire_write_data_array(ASR_IIC_ADDR_CHANGE_ADDR, [new_addr],1)
        self.address = new_addr
        return 1
    else:
        return 0
```

6) After instantiating the voice interaction module class in the main function, a list is defined to store the entries to be broadcast. Each element in the list is a tuple containing the type and ID of the entry to be played.

An `ASR_ANNOUNCER` value of `0x00` indicates that the entry type is a command prompt. Writing `0x01` to the register address `ANNOUNCER` triggers the broadcast of the corresponding entry: "**Going straight**".

An `ASR_ANNOUNCER` value of `0xFF` indicates a general broadcast message. Writing `0x01` to the `ANNOUNCER` register in this case corresponds to broadcasting the message: "**Recyclable waste**".

```
# Define the broadcast content and its corresponding ID (定义播报内容及其对应的ID)
announcements = [
    (ASR_CMDMAND, 1),  # Going straight (正在前进)
    (ASR_ANNOUNCER, 1),  # Recyclable waste (可回收物)
]
```

Then, the main function enters a `while` loop that repeatedly executes the following actions:

First, it calls the `sleep` function to delay for 1 second. Then, it prints a string to notify the user that the module's I2C address will soon be changed. The `ChangeAddr` function of the `asr` object is called with the new I2C address `0x34` passed as an argument.

The `ChangeAddr` function checks whether the input address is valid—specifically, whether it is either `0x33` or `0x34`. If the address is valid, the function returns `1` and begins communication to send the data.

An `if` statement is used to check whether the `ChangeAddr` function executed successfully. If so, the program delays for 100 ms to allow the new address configuration to take effect. It then communicates with the module using the new I2C address to trigger voice broadcasting of the entry.

```
        time.sleep(1)

        print("now,asr module's IIC Address is 0x34!")
        if(asr_module.ChangeAddr(0x34)):
            print("Success!")
            time.sleep(0.1)

            for cmd, id in announcements:
                asr_module.speak(cmd, id)
                time.sleep(2)

        else:
            print("Fail!")
```

Next, the program proceeds to change the I2C address of the voice interaction module back to `0x33` and uses this address to control broadcasting again. The process is nearly identical to the one described above and will not be elaborated here.

For detailed data lists, please refer to the "**5.1.3 CI1302_English_V00916_UART1_115200_2M**" located in the directory of **5. Firmware Development**.